PAPER

Phys. Educ. **59** (2024) 045010 (16pp) iopscience.org/ped

Fostering scientific methods in simulations through symbolic regressions

Fabio Llorella[1], **José Antonio Cebrián**[1], **Alberto Corbi**[2,*] **and Antonio María Pérez**[1]

[1] ESIT, Universidad Internacional de La Rioja (UNIR), Logroño, Spain
[2] Research Institute for Innovation & Technology in Education (UNIR iTED), Logroño, Spain

E-mail: alberto.corbi@unir.net

Abstract
Two-dimensional computer and tablet PC physics simulations have proved to be effective in helping students understand the fundamental principles of physics and related natural processes. However, the current approach to using these simulations tends to follow a repetitive cognitive and procedural pathway, regardless of the specific physical concepts being explored or software environment being used. This approach involves manipulating the simulation interface and collecting data through interaction with controls, widgets, or other contextual elements. Students then attempt to determine how these experimental measurements align with established laws, interactions, or mechanisms, as the teacher might have previously explained. We believe that this approach, while appropriate for education, obscures scientific processes, mainly related to the hypothetico-deductive model. To address this issue, we have developed a simple and adaptable computer environment that makes use of genetic algorithms (GAs) and symbolic regression to derive many of the basic laws of nature from the data collected by students using the popular physics education technology (PhET) simulations environment. Our proposal enables learners to observe how the order and relationships of mathematical tokens are routinely refined as new data points are added to the simulation setting. This iterative distillation technique can also be augmented with the interplay of dimensional analysis. In contrast with other more sophisticated artificial intelligence patterns, GA fit into the realm of grey box machine learning models. These type of evolutionary algorithms achieve the sought results by evolving mathematical models on each stage in an understandable way, which mimics the way

* Author to whom any correspondence should be addressed.

© 2024 IOP Publishing Ltd
All rights, including for text and data mining, AI training, and similar technologies, are reserved.

scientific breakthroughs are accomplished (over the course of generations of researchers and based of prior knowledge). By implementing this innovative approach, we can provide students with a more authentic empirical experience that fosters a deeper understanding of the principles of science and scientific discovery. Field tests with students supporting this claim have also been carried out.

Keywords: symbolic regression, genetic algorithms, physics simulations, grey box models, scientific methods

1. Introduction

Computer simulations have revolutionised the way in which students learn physics in K-12 and high school settings [1]. These simulations have provided a cost-effective, practical, and realistic laboratory experience for students, which has greatly enhanced their understanding of complex physical concepts [2, 3]. Moreover, they have made it possible to teach specific physical processes that are otherwise impossible to implement in real-life scenarios or lab settings.

One of the most prominent examples of scientific and math simulations in education is, undoubtedly, physics education technology (PhET) [4]. This non-profit project was initiated by the University of Colorado Boulder in 2002 and has become the go-to platform for physics, chemistry, and math simulations in various educational settings. PhET has achieved this status by providing open and standards-based technological infrastructure, primarily comprising HTML5, JavaScript, and CSS, which has greatly democratised access to empirical lab activities. Besides, the fact that PhET is based on web technologies has enabled the client–server architecture described in section 4.3.

Despite the numerous benefits of simulations, they all operate in a similar manner. In their traditional approach, a given law of nature (gravitation, sum of forces, buoyancy, etc) or a scientifically established definition (density, area, etc) is hardcoded into the simulation's internal programming and is seldom disclosed to the learner in order to make it more playful and engaging (gamification [5]). This method is also employed by PhET, which has led to its widespread popularity among educators and students alike. The students just have to tinker with various parameters and see how they affect the outcome of a physical phenomenon. In other words, all the theoretical background information is left to the teacher or external sources, allowing the student to focus on exploring and experimenting with the simulation. In this scenario, students can change the values of variables such as mass, speed, time, distance, etc by manipulating knobs, widgets, and other elements, but how these changes contribute to build scientific knowledge (i.e. a symbolic relation) is almost never shown.

Once the aforementioned variables have been adjusted by the learner, the simulation programmatically applies the relevant physical laws and displays the result in real-time (animations, sounds, messages, change in the status of specific objects/parts, information boxes, etc). This reactive process follows established didactic principles [6] and ensures that the student can see the cause-and-effect relationship between the shifted values and the outcome. This makes the learning experience both informative and enjoyable. Thanks to modern hardware, simulations like PhET can be run seamlessly on PCs, tablets, and even smartphones. This ensures that students have access to these tools wherever they are, making learning physics more accessible than ever before.

Although lab work is often seen as an essential aspect of education, it is not a perfect reflection of scientific methods. In true scientific practice, an hypothesis is formulated based on observable data, without knowing the outcome beforehand. The scientific process involves trial and error, and relies on previous knowledge about magnitudes and their mathematical relationships. Empirical data might be collected over the years (even generations of multiple researchers) and it is then used to refine a sought mathematical model of reality (figure 1). Nevertheless, modern statistical and computer methods like machine learning (ML), artificial intelligence (AI), genetic algorithms (GAs), and symbolic regression (SR)

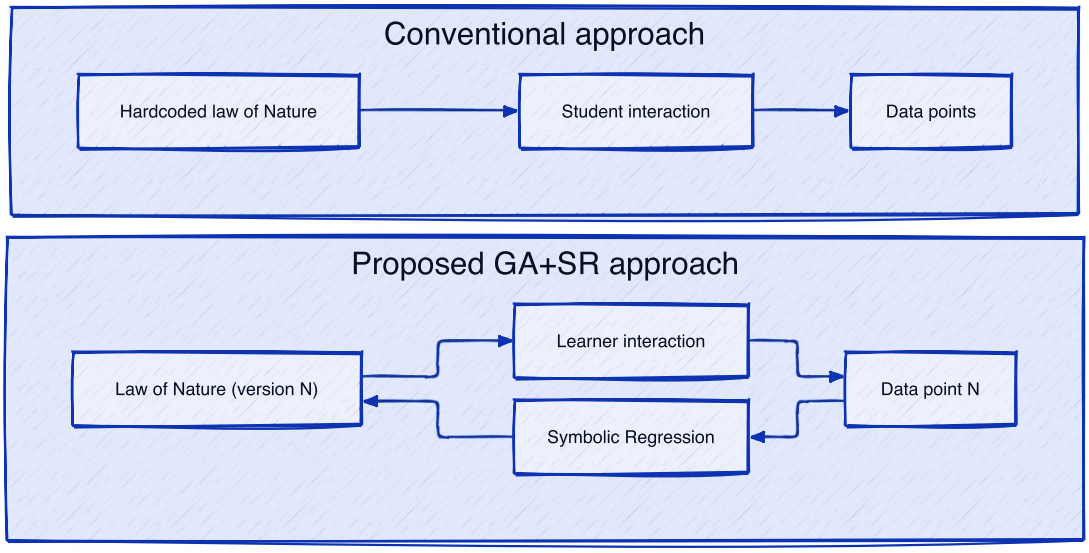

Figure 1. Mainstream/conventional way of student interaction with PhET simulations and the proposed approach in this paper based in the interplay of symbolic regression and genetic algorithms.

can significantly speed up the process. What once took decades of research can now be done in mere minutes.

This research presents a framework that integrates GA and SR with PhET simulations to accurately extract physical relationships from real-time data collected by students. The GA+SR approach was chosen for its simplicity, transparency, and close association with other science, technology, engineering and maths (STEM) fields such as biology and ecology, which are frequently taught in secondary education curricula. These type of evolutionary algorithms achieve the sought results by evolving mathematical models on each stage in an understandable way, which mimics the way scientific breakthroughs are accomplished (over the course of generations of researchers and based of prior knowledge).

The present article explains the workings of GA in greater detail in section 2. Moreover, in section 3, we vindicate the complementary nature or GA and SR, and its potential to resolve relatively complex regression issues. We briefly contrast our approach with more sophisticated AI-related ones as well.

The primary contribution of this study is an open and user-friendly computer framework that can generate mathematical expressions from a set of observations obtained from either physical experiments or human-defined scientific concepts, both simulated in PhET-based environments, contributing to the understanding of scientific processes among students. We chose PhET simulations due to their accessibility and flexibility, as many of them enable external access to read and modify their state, in addition to their popularity and fun nature.

2. GAs in physics

GAs belong to the realm of evolutionary algorithms and *grey box models* due to their unique position between the extremes of *white box* and *black box* ones [7]. GA, while not requiring a comprehensive understanding of the system's internal mechanisms, do rely on the principles of *natural selection* and *genetic variation*, allowing users to encode some domain-specific knowledge into the algorithm's design. Similarly, SR operates by discovering mathematical equations that represent the relationships within a dataset, blending known mathematical concepts with *data-driven discovery*. In both cases, these techniques incorporate a degree of human or domain expertise while still allowing for the exploration of complex systems with partially understood or *abstracted*

internals. This balance between transparency and abstraction makes GAs and SR powerful tools for modelling real-world systems in situations where a full white box approach may be impractical or a complete black box representation may lack the necessary interpretability. For these same reasons, GA turn out incredibly pertinent in K-12 domains.

Even though they were originally developed as a companion AI method for solving problems in biological evolution [8], GA have also had a large tradition of use in several fields of Physics. For instance, GA have already been applied in astrophysics for the study of cosmic ray propagation [9]. In [10], their authors use GA for the quantification of meteoroid fragmentation cascades (starting from their initial masses and velocities). Weather forecasting models such as the *weather research and forecasting* have also benefited from the application of GA for the quantification of wind power density and solar irradiance [11]. In the field of physics of materials, GA have played a major role in the search for optimised photonic crystal-based structures [12] and in hot compression tests for aluminium alloys [13]. These use cases represent only a small subset of the great potential that GA have to offer to the domain of physics research.

All these applications of GA share a common thread: they harness experimental data to deduce accurate symbolic outcomes. Essentially, GAs represent a collection of optimisation techniques that commence with observed data points and a limited understanding of the underlying physical variables.

In a more detailed explanation, a GA starts with an initial population of individuals, representing input data. These individuals undergo a process of evolution driven by natural selection. Based on random variations applied to the collective dataset, each individual is either chosen to progress (exhibiting endurance) or face extinction. This iterative process yields a fresh set of robust organisms, with a possibility of mutations occurring within them. Subsequently, individuals engage in crossbreeding and random mutation, giving rise to a new generation. As the process continues, it gradually converges towards an optimal solution for the problem at hand. This inherent simplicity, and the associated speed, make GA an invaluable tool for optimising challenges not only in the field of physics but also across various scientific disciplines. A global outline of the operation of GA is shown in section figure 2.

The main components and steps of GA are:

- As in nature, *chromosomes* represent the potential arrangement of *genes* utilised in the evolutionary process. Each gene entails a variable or an integral part of the equation governing the physical process (hidden in the programming of each PhET simulation, as commented in section 1). Chromosomes undergo then an iterative fusion and mutation process to give rise to new solutions. This is achieved through the application of genetic operators, tackled below. In the case of SR, this set of genes can be represented as trees of symbols (explained in section 4.1).
- A *mutation* occurs when we change a node of our algebraic expression tree. If we have a tree (chromosome) that represents the following algebraic expression: $m \times a$, a possible mutation could be to change the mass m by a and we would obtain the expression $a \times a$. Through mutation, it is possible to explore different areas of the search space while avoiding falling into a local minima. In our specific implementation, the mutation operator (figure 3) works in the following manner. A random number between 0 and 1 is generated. If that number is greater than a given mutation rate (given as an initial condition), then the node is transformed into another item randomly (which can, in turn, be a symbol or an operation). This simple procedure allows us to look for all possible solutions in the search space.
- A *crossover* operation allows us to combine two different algebraic expressions in order to obtain another that tries to maintain the benefits of the previous two. If we have a tree of the expression $m \times a$, and another representing $a + m$, a possible crossover could be $(m \times a) + m$ or $a + (m \times a)$, which would generate two different solutions. Crossover also allows us to move through the search space, but unlike mutation, the movement is more guided and it is still possible to fall into a local minima. For this

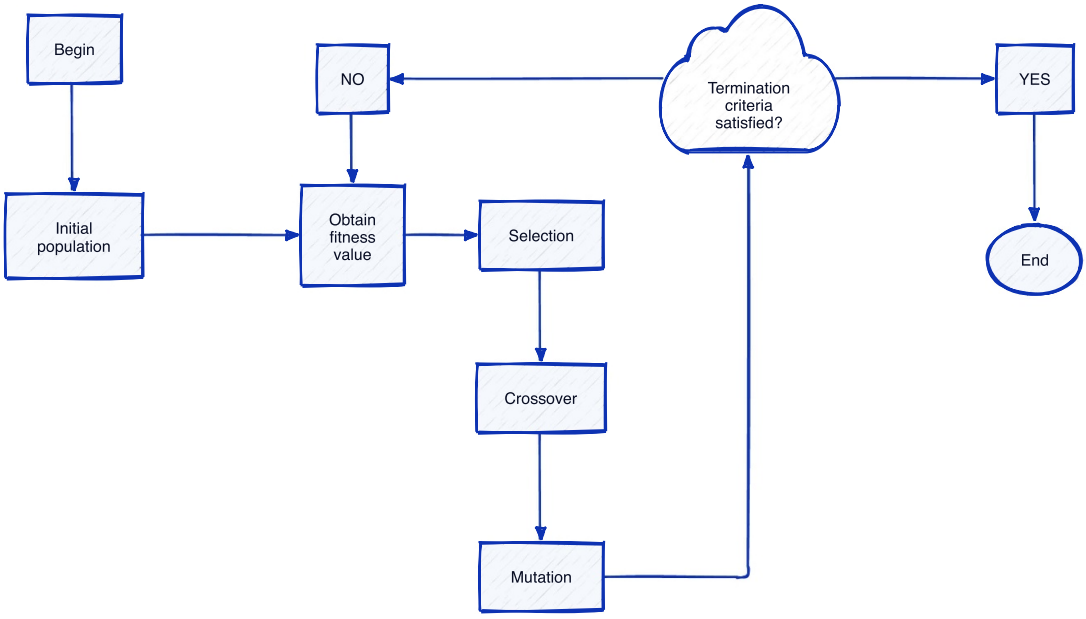

Figure 2. Outline of how a GA works.

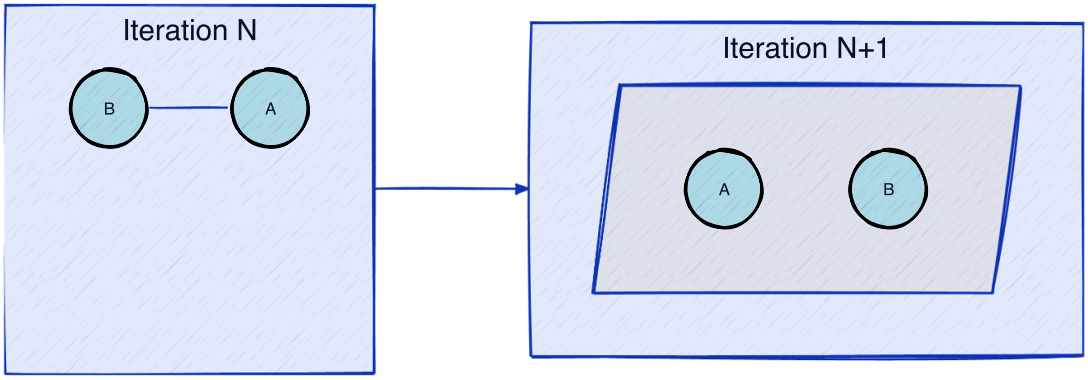

Figure 3. Mutation example. It can be seen that the node containing the sum function becomes another function (i.e. a product).

reason, the two operations (mutation and crossover) are usually applied in combination.

- To start the population, we usually resort to the random generation of algebraic expressions. In the case of physics, we can also take advantage of the fact of the *homogeneity* of the units (tackled in section 4.2). This trick has been used in our custom GA implementation and clearly contributes to accelerating the distillation of the final (and correct) solution.
- We finish the algorithm when the error is less than a given threshold (which is 10^{-17} for our own custom implementation). We then return the achieved symbolic expression as the best outcome possible.

3. Use of GAs in STEM

GAs and SR offer innovative and effective ways to teach STEM-related subjects in schools, fostering a deeper understanding of complex scientific concepts and problem-solving skills. These methodologies can be particularly valuable

in engaging students and preparing them for real-world challenges in science, technology, engineering [14], and mathematics [15].

Firstly, GAs can be employed to illustrate principles of optimisation and search algorithms. Students can learn how GAs mimic the process of natural selection and evolution to find optimal solutions to problems. For instance, they can explore how GA can be used to optimise parameters in physics experiments or engineering designs [16]. By actively participating in the creation and modification of GAs, students gain hands-on experience in programming and algorithm design, thereby enhancing their computational and mathematical skills [17].

SR, on the other hand, offers a powerful tool for exploring mathematical modelling and equation discovery. In STEM subjects like physics [18] and engineering [19], students often encounter complex relationships between variables. SR allows them to uncover these relationships by automatically generating mathematical expressions that fit empirical data. By applying SR, students can gain a deeper insight into the mathematical underpinnings of scientific phenomena, reinforcing their understanding of key concepts in algebra, calculus, and statistics.

Furthermore, the combination of GAs and SR provides a holistic approach to solving STEM problems. Learners can use GA to optimise the parameters for SR models and fine-tune the equations that best fit experimental data. This integrated approach not only strengthens their computational skills but also emphasises the importance of interdisciplinary thinking, a crucial aspect of modern STEM fields where complex problems often require expertise from multiple domains.

Certainly, there exist other more eminent, powerful and even more accurate AI methods for performing SR, but they might turn out too *black box* and computing power demanding to be implemented by secondary school students. For instance, [20] uses a novel generative transformers model generative pre-trained transformer (GPT) to carry out SR, but according to our tests, a (neural) GPT-based implementation requires twice the memory and time to distil the same K-12 physics laws examined in this work and listed in section 5.1. According to our experiences, it *feels* more natural to use GAs than neural networks for simple SR problems (like the ones present in any K-12 curriculawith just 2–5 variables).

In summary, the interplay of GAs and SR offer an engaging and practical way to teach STEM subjects in schools. By integrating these computational techniques into the curriculum, educators can empower students to become proficient in problem-solving, algorithm design, mathematical modelling, and interdisciplinary thinking. These skills are highly valuable in the ever-evolving world of science and technology.

4. Physics simulations and SR

As addressed in section 3, GA and SR are two powerful computational techniques that have gained significant attention in the field of ML. In recent years, the demand for accurate regression models that can effectively capture the underlying relationships between input variables and target outputs has grown substantially. The tandem composed of GA and SR offers promising solutions to this challenge.

In our approach, we combine GA with SR to uncover the correct mathematical relation (i.e. a physical law) by searching through a *space of mathematical building blocks* (a multilayer binary tree of symbols). This enables the discovery of interpretable equations that capture complex relationships in the data.

Certainly, there already exist several software tools to work with SR. Perhaps, the most reputed ones are Eureqa and TuringBot [21]. However, we have chosen to create our own open source framework from scratch using the high-level programming language Python. We think that a custom and open development may allow the students to understand the computational and algorithmic underpinnings of GA and SR. Although not our main goal, the most capable students, if so desired, will be able to read the code or even modify it at will. This source code is hosted at https://github.com/pammacdotnet/GASR. As stated in section 2 and later detailed in section 4.2, this custom implementation makes intensive use of dimensional analysis (DA) to speed up calculations.

An open architecture has also allowed the interplay with existing PhET simulations (as explained in section 4.3). Our pluggable solution

has no other limitations than having a local (or even a cloud-based) Python runtime installed, together with the basic libraries that usually come as part of a typical Python-based environment:

numpy, for basic vector operations.
tree, for tree-like information storage.
pickle, for serialisation and basic output.

The structure of our genetic program can be summarised in the following pseudocode:

```
Initialise the population with random
 candidate solutions
Repeat until a termination condition
 is met:
   Evaluate fitness of each candidate
   Select parents for reproduction
    based on their fitness
   Generate new offspring through
    crossover and mutation operations
   Replace some members of the
    population with the new offspring
Return the best solution found in the
 population
```

For the sake of completeness, we have developed (and tested, with very similar results) another implementation of our approach using the PySR Python package [22]. PySR is an open-source tool for SR, an ML task where the goal is to find an interpretable symbolic expression that optimizes some target function. It is also partly written in the Julia programming language. PySR is very well optimised and does not need to rely on DA. However, in contrast with our own development, PySR is a very complex and extensive Python library that does not easily allow its study by young curious physics learners. It is is written by an international team of computer experts and scientists and has therefore been exhaustively polished over the years.

As explained in section 4.3, both frameworks (our custom one and PySR) share the exact same goal and can be used interchangeably (on modern hardware) with any of the PhET simulations listed in table 2. Perhaps, the only difference is that ours also allows the tangential teaching of the concept of dimensional homogeneity.

4.1. Algebraic expressions as mutable binary trees

Algebraic expressions consist of characters representing constants, symbols, or functions. In turn, operations can be classified into unitary or binary based on the number of arguments they take. Understanding the anatomy of algebraic expressions is vital in the context of SR, where this structure corresponds to the previously discussed chromosome (which starts with a random configuration).

In the context of GAs and SR, chromosomes are often represented as mutable, multilayered binary trees. This representation is highly suitable for evolving mathematical expressions and models, allowing GAs to search for the most appropriate mathematical formula to fit a given dataset. The main components and steps are:

Nodes and edges represent mathematical operations (functions) or variables. Each node has zero or more children, depending on the arity of the function it represents (number of arguments or operands that a mathematical function or operation takes).

Root node is the main operation in the expression: addition, multiplication, a trigonometric function, or any other mathematical function.

Intermediate nodes are the mathematical operations or functions, and they can have child nodes that are either variables or other intermediate nodes. The tree structure enforces the order of operations, ensuring that expressions are well-formed mathematically.

Leaf nodes are variables or constants. Variables correspond to the input features or independent variables of the problem, while constants are fixed values.

Tree depth can vary, allowing for the representation of complex mathematical expressions. GAs can evolve these trees by changing the structure of the tree (e.g. adding, deleting, or swapping nodes) and by modifying the functions and variables at each node.

Fitness evaluation is typically determined by how well it fits the given dataset. This involves evaluating the tree for each data point and comparing the results to the actual values. The closer the tree's output is to the target values, the better its fitness.

$$3 \times x^2 + 2 \times x \quad (1)$$

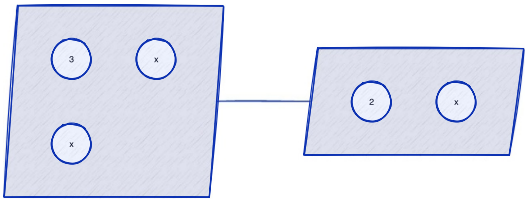

Figure 4. Parts of a mathematical expression and the tree (chromosome) representing it.

```
[
 "Add",
 ["Multiply", 2, "x"],
 ["Multiply", 3, "x", "x"]
]
```

```
3x^2+2x
```

```
<math><mrow>
  <mn>3</mn><msup>
   <mi>x</mi>
   <mn>2</mn></msup>
  <mo>+</mo>
  <mn>2</mn>
  <mi>x</mi>
</mrow></math>
```

Figure 5. Three ASCII representations (MathJSON, AsciiMath, and MathML, respectively) of the chromosome associated to equation (1).

Genetic operations use standard genetic operations, such as crossover (recombining subtrees of two parent trees) and mutation (changing a node or subtree), to evolve the binary trees over generations. These operations aim to create new expressions that are potentially better at fitting the data. In this work, the only genetic operator that we have used has been the mutation operator (figure 3). The reason for this is that, according to our tests and experiments, other operators such as crossover, did not specially contribute to the efficiency (time taken) of our approach. We also verified that DA (discussed in section 4.2) already outperformed crossover by itself. This decision allowed us to develop a cleaner, simpler and more efficient code.

Termination condition. The evolutionary process continues until a termination condition is met, which could be a maximum number of generations, a fitness threshold, or other criteria.

Binary tree schematic representations are advantageous because they can capture a wide range of mathematical expressions, from linear models to highly complex nonlinear equations (figure 4). In our implementation for secondary school students, we have chosen the following criteria:

- Symbols (variables or constants) appear as circles or ovals.
- Two or more symbols connected by a straight line entail addition between them. In order to express subtraction, a minus sign can be prefixed to the ones required.
- Two or more symbols inside a parallelogram involve multiplication. Division can be denoted with a negative exponent.

All trees in this paper have been drawn following the algorithm discussed in [23].

The generated trees can be serialised using text- or web-based standards as well. Some of them are: MathJSON, MathSON, AsciiMath [24] or MathML. For instance, the examples in figure 5 use MathJSON (proposed by the project CortexJS) and AsciiMath for serialising equation (1) to MathML.

Of course, trees can also be expressed as LaTeX equations, which will come in handy when

being rendered in web browsers, as explained in sections 4.3 and 5.2.

As stated above, binary trees (our chosen representation of a chromosome) and mutations make up the foundation of our research: finding the best mathematical expression from a set of observations carried out in a simulation environment (such as PhET).

4.2. DA

As already explained in section 4, in contrast with the PySR framework, our custom GA+SR implementation makes use of dimensional analysis to speed up calculations. As widely known, in physics, every mathematical expression must comply with the principle of dimensional homogeneity. This tells us that the left part of an equality must have the same dimensional units as the right part. For instance, the following expression:

$$\frac{\mathrm{kg}\cdot\mathrm{m}}{\mathrm{s}^2} = \mathrm{kg}\cdot\frac{\mathrm{m}}{\mathrm{s}^2}, \tag{2}$$

satisfies this principle, since the two parts of the equality have the same units (representing a force). However, this expression:

$$\frac{\mathrm{kg}}{\mathrm{s}} = \frac{\mathrm{kg}^2}{\mathrm{s}^2}, \tag{3}$$

does not comply with the principle of dimensional homogeneity, since the dimensions of the left part of the equality are different from the right part. This simple and well-known principle can speed up the GA developed as part of this work. Without dimensional analysis, we start with a given arbitrary binary tree shape and an associated square error. If this distance (relative to the *correct chromosome*) is large, this tree will be surely discarded in the next iteration. However, those trees with small error (but still wrong) will be taken into account, since the distilled formula starts to resemble the final one, turning the process slower than necessary.

Nevertheless, this randomness can be further tailored if dimensional analysis is added the mix. If we initialise the initial population (first generation) with an equation that already meets dimensional homogeneity (a somewhat well scaffolded tree), the search space is narrowed and the risk of falling into local minima is smaller. This does not entail that dimensionally incoherence may re-emerge in subsequent iterations. This can be noticed in figure 10, where the first iteration outputs a constant with the expected density dimensions, but the second and third ones clearly violate again dimensional homogeneity (the rest of the constants have other dimensions or are dimensionless).

Table 1. Dimensions, units, and their codification as prime numbers.

Dimension	SI init	Prime
Length	meter (m)	3
Time	second (s)	5
Amount	mole	11
Current	ampere (A)	13
Temperature	kelvin (K)	17
Light intensity	candela (cd)	19
Mass	kilogram (kg)	7

Our computer implementation of DA has been based on the *fundamental theorem of arithmetic* and use of prime numbers associated to each of the seven SI fundamental units listed in table 1.

This trick enables the codification of new derived units (energy, force, etc) as the product of prime numbers. For example, $\mathrm{kg}\,(7)\cdot\mathrm{m}\,(3)$ would be coded by the number 21 and $\mathrm{kg}\,(7)\cdot\mathrm{m}\,(3)\cdot\mathrm{s}\,(5)$ with 105. In the case of fractions, the numerator and denominator are composed (and tested) separately.

4.3. Connection to PhET

The PhET project also provides a protocol extension known as PhET-iO [25, 26]. Simulations that adhere to these guidelines are enhanced with a set of interoperability features that enable third parties to customise, integrate, and monitor [27] students while interplaying with PhET animations. With PhET-iO, educators and developers can capture data on user interaction to assess student achievement, power personalised or adaptive-learning systems, or distil teacher reports.

In our case, a web client–server architecture has been chosen. Figure 6 shows a diagram of our GA+SR implementation, where each learner

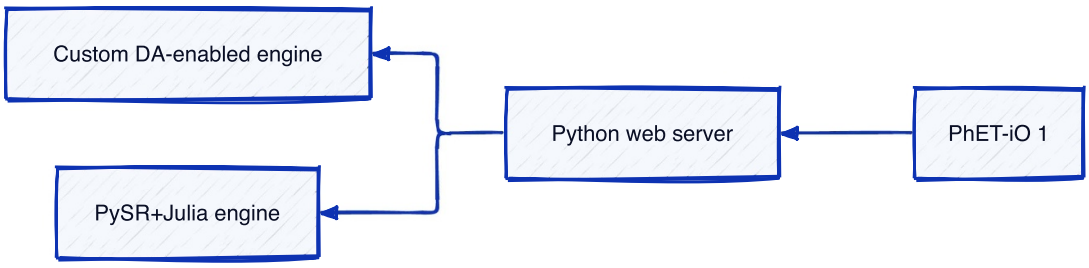

Figure 6. Client–server architecture for our specific GA+SR implementation. The computing backend can either be based on the PySR+Julia tandem or on our custom DA-enabled runtime.

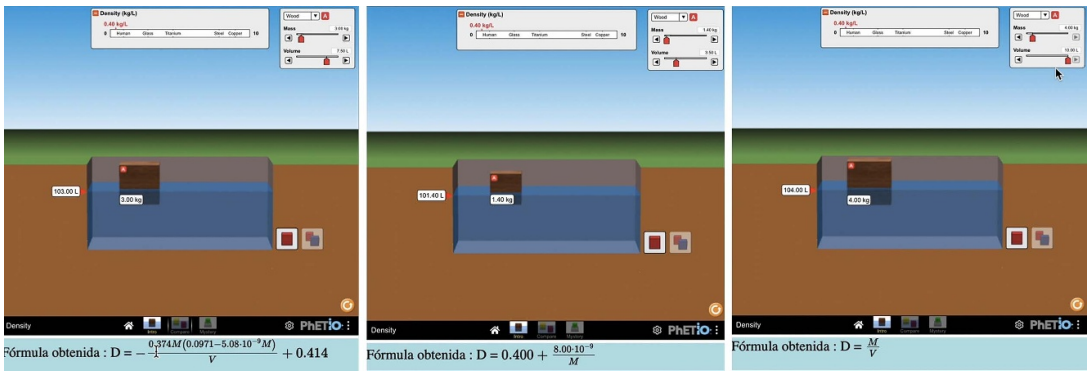

Figure 7. Several screenshots of the PhET density simulation linked to the GA+SR engine introduced in this work.

is running a PhET instance in his/her own web browser. However all computations (for all connections) are being carried out (separately) in a central machine or virtual service. The backend service can either be based on the PySR+Julia tandem or on our custom DA-enabled runtime, no matter the PhET simulation. For the sake of diversity, in section 5.2, both approaches have been tested.

In turn, the web request broker and the GA+SR engine can run in isolated virtual environments. Virtualisation techniques are taking over the whole computing industry, and the computational labs realm is no exception [28].

With more detail, when a student changes any value in a simulation, new data points are transmitted (using an HTTP POST request) to our backend GA+SR service. This engine takes all inputs into account and a computed LaTeX formula is sent back to the student's browser. This mathematical expression is then rendered in the learner's screen using web-based technologies such as MathML, KaTeX or MathJax.

In figure 7, it is shown several screenshots of the PhET density simulation (whose field test with students is later tackled in section 5.2) and how the aforementioned math expressions are displayed on quasi real-time as the learner plays with knobs, objects, drop-down menus, etc.

Not all PhET sims are PhET–iO enabled (this comprises other nice third-party simulation solutions, such as EJS [29]). In this scenario, students can still manually transfer the obtained inputs to a simple external mobile, desktop or web interface that, in turn, is connected to our GA/SR engine (where all calculations take place). For instance, for the Gravity Force Lab sim, we have chosen to build a custom accompanying web application (figure 8) which, for the sake of completeness, interfaces with a PySR-based, DA-less instance of our solution.

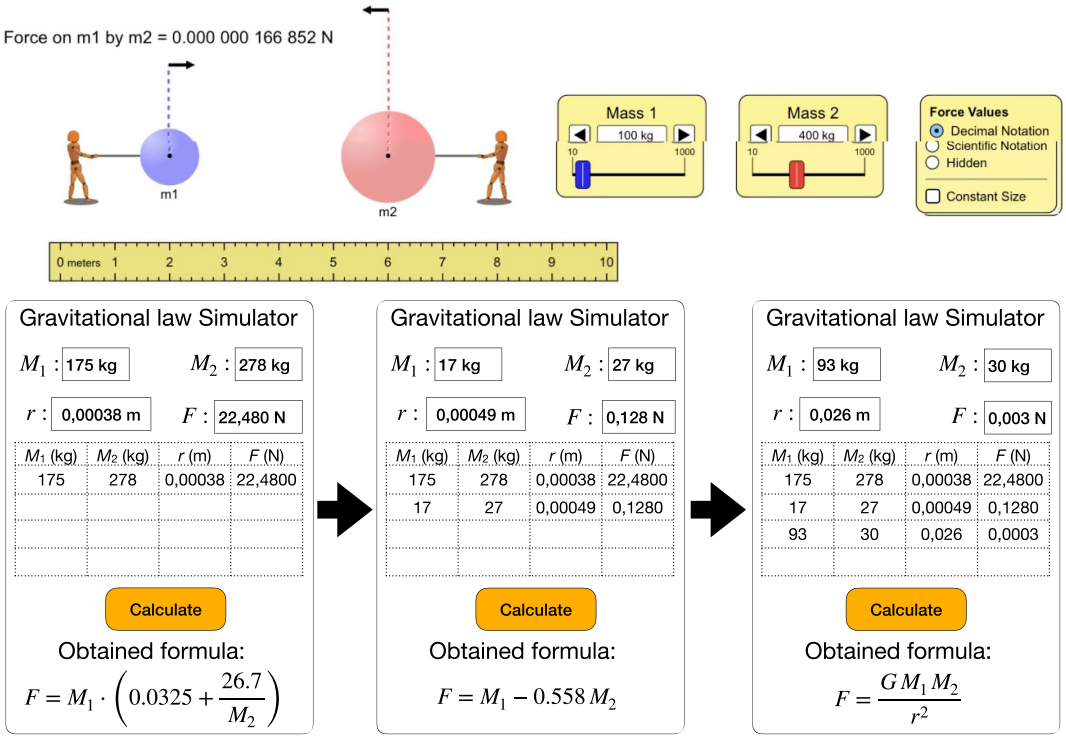

Figure 8. PhET's gravitational force sim (top) and the custom-built student's external mobile/web app tool for manually gathering data (bottom). Three iterations are needed until the termination criteria is satisfied and the correct Newton's equation is obtained. Learners manually introduce the gathered empirical values when playing with the controls: the two masses (M_1 and M_2), the distance between them (r) and the interplaying force (F), given by PhET as a result. In this case, the more efficient PySR+Julia backend was used (not requiring dimensional analysis). For the sake of simplicity, the gravitational constant G was hardcoded into the program's source (i.e. it is not distilled as part of the iterative process).

5. Results and discussion

Our main results have to do, on the one hand, with the analysis of which PhET simulations can technically (and *de facto*) benefit if they are connected to our AI service. On the other hand, we questioned ourselves how a real set of learners would feel while using enhanced PhET sims that constantly update (and display) the mathematical expression behind them (i.e. would they gain a more intuitive insight about the concepts of regression,DA and scientific methodologies?).

5.1. SR and PhET simulations

With our custom made DA-enabled GA+SR implementation, we are able to satisfactorily derive 15 symbolic expressions from the PhET Simulations Physics sub-directory. It might not seem a lot, but they cover almost all areas of a physics K-12 curricula: mechanics, matter and electromagnetism. These well-know expressions are listed in table 2.

Regarding our own GA implementation, the symbolic expressions that have unquestionably benefited from DA are Newton's law of universal gravitation and Coulomb's inverse-square law (table 3). In the first case, the reason has to do with the non-linearity of the law, together the weak value of Cavendish's gravitational constant (we quickly fall into a local minima). Regarding electric force, the reason behind is the general small value of electric charges. Certainly, both scenarios might be addressed by scaling the data,

Table 2. Examined PhET simulations where GA have proven to work.

Cat.	Sim name	Sim URL	Studied physics law
Mechanics	Masses and Springs	masses-and-springs	Hooke's law: $F = -k\Delta x$
	Gravity force	gravity-force	Law of gravity: $F = \frac{-GmM}{R^2}$
	Projectile motion	projectile-motion	y-axis motion: $y = y_0 + v_0 t + \frac{1}{2}at^2$
Matter	Gas intro	gases-intro	Ideal gas law: $PV = nRT$
	Density	density	Density: $\rho = \frac{M}{V}$
	Under pressure	under-pressure	Hydrostatic pressure: $P = \rho gh$
Electromagnetism	Coulomb's law	coulombs-law	Coulomb's law: $F = \frac{KqQ}{R^2}$
	Charges and fields	charges-and-fields	Electric potential: $V = \frac{Q}{R}$
	Capacitor lab	capacitor-lab-basics	Capacitance: $C = \frac{Q}{V}$
	Ohm's law	ohms-law	Ohm's law: $I = \frac{V}{R}$
	Resistance	resistance-in-a-wire	Resistance: $R = \frac{\rho L}{A}$
	Color vision	color-vision	Freq.: $\omega = 2\pi f$ and wave number $k = \frac{2\pi}{\lambda}$

Table 3. Time taken (in seconds) with DA enabled and disabled (~~DA~~) to distil the correct symbolic expression. Hardware used was a dual Intel Celeron processor clocked at 2.60 GHz and 8 Gb of RAM.

Physical expression	~~DA~~	DA
Gravity: $F = G\frac{m_1 m_2}{d^2}$	$\sim\infty$	5
Coulomb's: $F = k\frac{qQ}{d^2}$	$\sim\infty$	10
Average value: $\bar{x} = \left(\frac{1}{n}\sum_{i=1}^{n} x_i\right)$	115	11
Potential energy: $E_p = mgh$	20	5
Resistance: $R = \frac{\rho}{A}l$	10	5
Kinematics: $y = y_0 + v_0 t$	65	65

but this falls out of the study of this work. In addition, our GA+SR execution does not work when the expression involves a delta, i.e. a span in magnitude such as Euclidean distance. This type of expressions do not allow us to obtain information on the underlying algebraic structure, causing the system to fluctuate out of control.

As discussed in section 4.2, a way to improve the SR process is using DA. This technique allows the GA algorithm to start with a symbol tree (only the first generation) that meets the correct physical dimensions and therefore will be closer to the optimal chromosome. The GA process is then left free again to randomly select new genes (with no further dimensional constraints).

Finally, as commented in section 4, another approach could be delegating the genetic/regression process to the reputed PySR module, which is more efficient than ours. This is exactly what we did for the field test involving Newton's law, tackled next.

5.2. Field tests with K-12 students

In order to test the presented if the discussed approach can really contribute to give more insight to young learners about scientific paradigms, two field tests have been carried out. Our study involved 360 K-12 students with no prior knowledge of PhET or Physics simulations. These young learners (figure 9) were introduced to PhET with the help of the density simulation and the Gravity Force Lab:

Density sim. In this case, the correct formula description can be attained with our own engine and by making use of DA. This also enables delving a little more into the concept of dimensional homogeneity (section 4.2) among students. Certainly, the concept of density is more of an *ad hoc* definition established by scientists than a true physical law. However, for a K-12 physics learner,

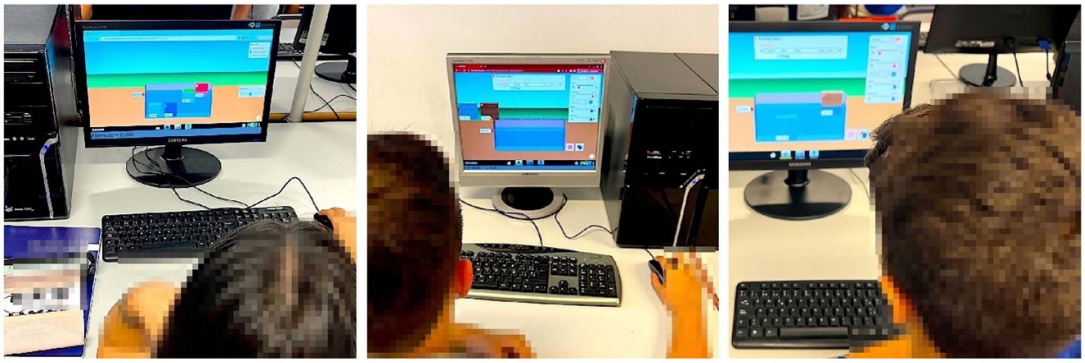

Figure 9. Students while enjoying a GA+SR-powered PhET simulation.

a symbolic definition is still a mathematical expression somehow connected to the physical world.

Newton's law. In this scenario, the more refined PySR framework is used (no DA).

Density simulation by PhET allows students to explore the concept of density, which is, as widely known, defined as the amount of mass per unit of volume. The simulation allows users to experiment with different objects of varying masses and volumes, and see how they interact with different fluids, such as water, oil, and alcohol. In the activity, learners can select different objects to drop into a fluid, and observe how they sink or float based on their density relative to the density of the fluid. They can also adjust the volume and mass of the objects to see how they affect the buoyancy of the object. Additionally, it is possible to alter the density of the fluid, and see how it affects the behavior of the objects inside it. Children are not given any previous information regarding the equation defining the concept of density: they just know it has something to do with two specific manipulable magnitudes such as mass and volume. It is the repeated (and playful) interaction with the simulation (and the aforementioned values) what ultimately leads to the unsupervised calculation (and display) of the correct mathematical relationship. As stated, the symbolic computation takes place within a continuous loop in our custom developed GA+SR and DA-enabled engine. This can help students better understand topics such as Archimedes' principle and buoyancy.

With more detail and as explained in section 4.3, when a student alters any of these parameters, a PhET-iO event is triggered, a POST request with the new data is sent to the GA+SR backend. Then, a new tree is computed (taking into account the added experimental points) and its associated chromosome is sent back to the learner's browser in the form of a appropriately LaTeX formatted expression. Figure 10 shows the evolution of this chromosome in all three representations (multilayered binary tree, symbolic expression and LaTeX formulae).

In the case of the Gravity Force Lab, which is not PhET-iO-enabled, students used a simple custom written web/mobile app where they had to manually transcribe the obtained inputs.

The results have confirmed that around 30% of the students did understand the concept of regression very well, while around 45% of them understood this idea moderately well. Besides, the vast majority intuitively understood the inner workings of GAs. In the context of the density simulation, when surveyed about the apprehension of the concept of dimensional homogeneity, the vast majority of the learners claimed that the experienced had contributed to better understand its significance and relevance.

We believe these outcomes would not have taken place with other AI methods. Neural networks, in particular, are excessively intricated in comparison with the more understandable (and yet efficient) grey box models. In general, learners participating in the experiments also commented that the presented computational setting (PhET + the GA+SR engine) helped them

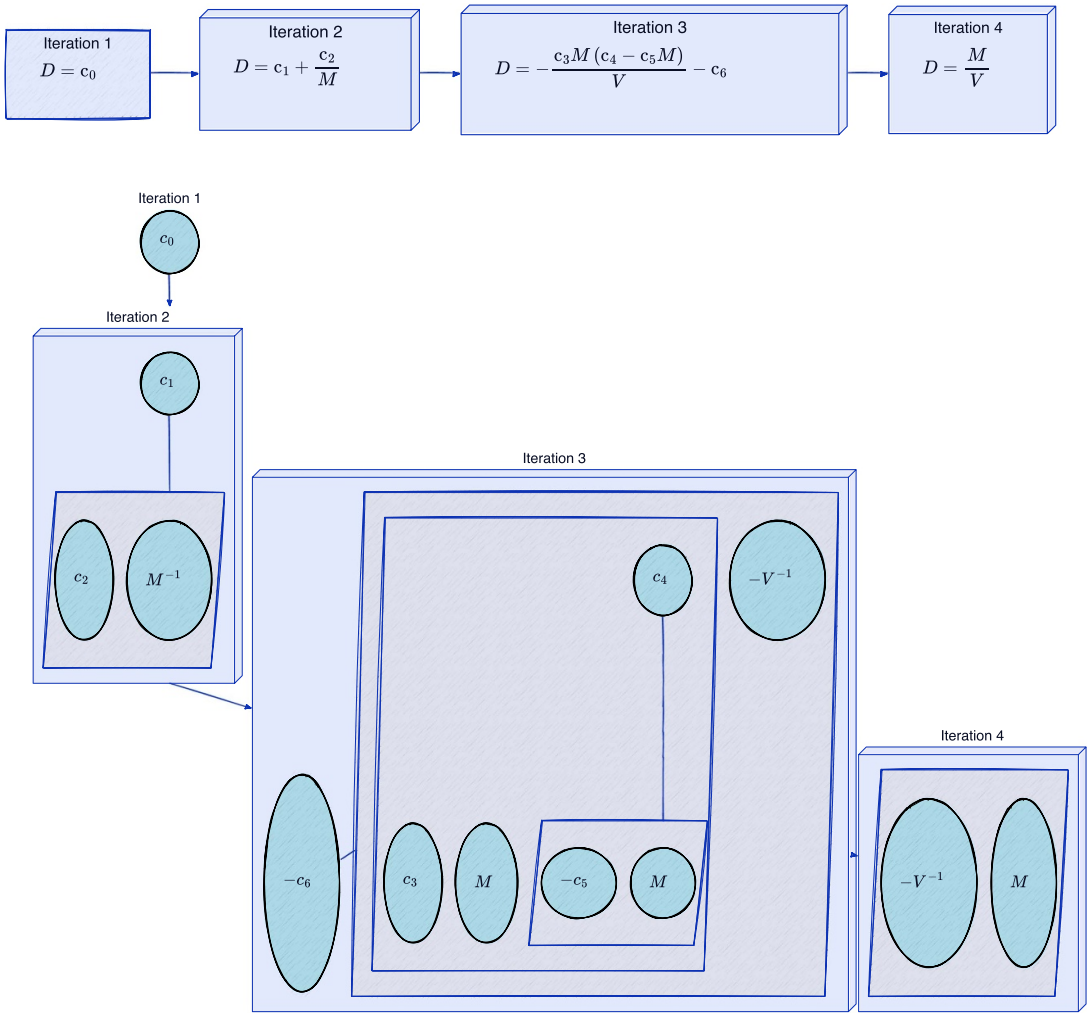

Figure 10. Example of the iterations of a GA during the distillation of the equation governing the concept of density. In this specific case, our custom GA+SR implementation was used and dimensional analysis was activated. In fact, the constant c_0 in the first iteration has the correct dimensions, ML^{-3}, but the rest (c_1, … ,c_6) have other units or are just dimensionless.

to better understand the underlying physical concepts (i.e. density and Newton's law).

6. Conclusions

In this work we have presented the scaffold of a web add-on for deriving the correct physical expressions from the experimental measurements compiled by students when playing with a set of PhET-powered simulations. Our solution is based on an SR approach (in turn, supported by GAs and dimensional coherence). All of these concepts can be easily recognisable by high school students and, at the same time, they enable a better understanding of the foundations of empirical scientific research and data-driven discovery. Two field tests with real learners has supported this initial hypothesis.

We have also explained the foundations of genetic programming and how it can be further refined in the case of simple physical laws by taking into account DA.

Future implementations of our work may comprise the use of the data points gathered by

all students in the same classroom. This alternative way of functioning would make them delve deeper into the fact that science is a collaborative effort and knowledge is built on cooperative experimentation.

Data availability statement

The data cannot be made publicly available upon publication because no suitable repository exists for hosting data in this field of study. The data that support the findings of this study are available upon reasonable request from the authors.

Acknowledgment

This research is partially funded by Universidad Internacional de la Rioja through the Research Institute for Innovation & Technology in Education.

Ethical statement

All field tests with students were carried out in accordance with the ethical policies outlined by the Spanish Ministry of Education. Informed consents from participants (and/or their legal guardian) were also obtained accordingly. Our work was conducted in agreement with the ethical principles required by the Editorial Board. Photos from students were also taken with their consent and awareness.

Participant consent statement

All participants (and/or their legal guardian) gave explicit consent for their participation in the present study.

ORCID iDs

Fabio Llorella https://orcid.org/0000-0002-7342-8054
José Antonio Cebrián https://orcid.org/0009-0007-9611-6102
Alberto Corbi https://orcid.org/0000-0002-7282-4557
Antonio María Pérez https://orcid.org/0009-0002-8854-3482

Received 17 October 2023, in final form 14 March 2024
Accepted for publication 9 April 2024
https://doi.org/10.1088/1361-6552/ad3cad

References

[1] D'Angelo C, Rutstein D, Harris C, Bernard R, Borokhovski E and Haertel G 2014 Simulations for STEM learning: systematic review and meta-analysis *Menlo Park: SRI Int.* **5** 1–5

[2] Tolga G O K 2011 The effects of the computer simulations on students' learning in physics education *Int. J. New Trends Educ. Implicat.* **2**

[3] James Banda H and Nzabahimana J 2021 Effect of integrating physics education technology simulations on students' conceptual understanding in physics: a review of literature *Phys. Rev. Phys. Educ. Res.* **17** 023108

[4] Yunzal A N and Casinillo L F 2020 Effect of physics education technology (PhET) simulations: evidence from stem students' performance *J. Educ. Res. Eval.* **4** 221–6

[5] Padilla-Zea N, Aceto S and Burgos D 2019 Social seducement: Empowering social economy entrepreneurship. The training approach *Int. J. Interact. Multimed. Artif. Intell.* **5**

[6] Bainomugisha E, Lombide Carreton A, van Cutsem T, Mostinckx S and de Meuter W 2013 A survey on reactive programming *ACM Comput. Surv.* **45** 1–34

[7] Babovic V and Keijzer M 2000 Genetic programming as a model induction engine *J. Hydroinform.* **2** 35–60

[8] Mitchell M 1995 Genetic algorithms: an overview *Complex* vol 1 (Citeseer) pp 31–39

[9] Luo X-L, Feng J and Zhang H-H 2020 A genetic algorithm for astroparticle physics studies *Comput. Phys. Commun.* **250** 106818

[10] Henych T, Borovička J and Spurný P 2023 Semi-automatic meteoroid fragmentation modeling using genetic algorithms (arXiv:2301.11076)

[11] Sward J A, Ault T R and Zhang K M 2022 Genetic algorithm selection of the weather research and forecasting model physics to support wind and solar energy integration *Energy* **254** 124367

[12] Sanchis L, Andreas Håkansson D L-Z'on, Bravo-Abad J and Sánchez-Dehesa J 2004 Integrated optical devices design by genetic algorithm *Appl. Phys. Lett.* **84** 4460–2

[13] Kronberger G, Kabliman E, Kronsteiner J and Kommenda M 2022 Extending a physics-based constitutive model using genetic programming *Appl. Eng. Sci.* **9** 100080

[14] Ahmad I, Shahzad M and Palensky P 2014 Optimal PID control of magnetic levitation

system using genetic algorithm *2014 IEEE Int. Energy Conf. (ENERGYCON)* (IEEE) pp 1429–33

[15] Schmidt M and Lipson H 2009 Distilling free-form natural laws from experimental data *Science* **324** 81–85

[16] Kiyani E, Shukla K, Em Karniadakis G and Karttunen M 2023 A framework based on symbolic regression coupled with extended physics-informed neural networks for gray-box learning of equations of motion from data (arXiv:2305.10706)

[17] Schmidt M and Lipson H 2009 Symbolic regression of implicit equations *Genetic Programming Theory and Practice VII* (Springer) pp 73–85

[18] Udrescu S-M and Tegmark M 2020 AI Feynman: a physics-inspired method for symbolic regression *Sci. Adv.* **6** eaay2631

[19] Lino A, Rocha A and Sizo A 2016 Virtual teaching and learning environments: automatic evaluation with symbolic regression *J. Intell. Fuzzy Syst.* **31** 2061–72

[20] Valipour M, You B, Panju M and Ghodsi A 2021 SymbolicGPT: a generative transformer model for symbolic regression (arXiv:2106.14131)

[21] Ashok D, Scott J, Wetzel S J, Panju M and Ganesh V 2021 Logic guided genetic algorithms (student abstract) *Proc. AAAI Conf. on Artificial Intelligence* vol 35 pp 15753–4

[22] Cranmer M 2023 Interpretable machine learning for science with PySR and symbolicregression (arXiv:2305.01582)

[23] Borgfeld Y 2019 Tool Support for Layout Algorithm Development with ELK *Master's Thesis* Kiel University

[24] Brunton K, McKain D, Bates S and Maciocia A 2006 Technical aspects of developing web-based Mathsaware assessment systems *Proc. CETL-MSOR Conf.*

[25] Moore E B and Perkins K K 2018 Advances in PhET interactive simulations: interoperable and accessible *Cyber-Physical Laboratories in Engineering and Science Education* pp 141–62

[26] Auer M E, Azad A K M, Edwards A and De Jong T 2018 *Cyber-Physical Laboratories in Engineering and Science Education* (Springer)

[27] López-Tavares D, Kauzmann M and Perkins K 2020 Visualizing student engagement with simulations: a dashboard to characterize and differentiate instructional approaches *Proc. Physics Education Research Conf. (PERC)* pp 579–84

[28] Corbi A, Burgos D, Franck Vidal F A and Albiol A 2019 X-ray imaging virtual online laboratory for engineering undergraduates *Eur. J. Phys.* **41** 014001

[29] Esquembre F 2004 Easy Java Simulations: a software tool to create scientific simulations in Java *Comput. Phys. Commun.* **156** 199–204